

Deep Q-Learning (DQN)

The problem of Q-learning

Q-learning exploits Bellman's equation

$$Q^*(s, a) = \mathbb{E}_{s'}[r_0 + \gamma \max_{a'} Q^*(s', a')]$$

We compute Q^* via iterative updates:

$$\underbrace{Q^{i+1}(s, a)}_{\text{next estimation}} \leftarrow \underbrace{Q^i(s, a)}_{\text{current estimation}} + \alpha \underbrace{(r_0 + \gamma \max_{a'} Q^i(s', a') - Q^i(s, a))}_{\text{recursive update}}$$

$Q^i \rightarrow Q^*$ when $i \rightarrow \infty$.

Not scalable!

Must compute $Q(s, a)$ for every state-action pair.



The problem of Q-learning

Q-learning exploits Bellman's equation

$$Q^*(s, a) = \mathbb{E}_{s'}[r_0 + \gamma \max_{a'} Q^*(s', a')]$$

We compute Q^* via iterative updates:

$$\underbrace{Q^{i+1}(s, a)}_{\text{next estimation}} \leftarrow \underbrace{Q^i(s, a)}_{\text{current estimation}} + \underbrace{\alpha(r_0 + \gamma \max_{a'} Q^i(s', a') - Q^i(s, a))}_{\text{recursive update}}$$

$Q^i \rightarrow Q^*$ when $i \rightarrow \infty$.

Not scalable!

Must compute $Q(s, a)$ for every state-action pair.

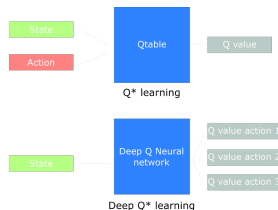
Deep Q-learning

Deep Q-learning: use a function approximator (a Neural Network!) to estimate the optimal action-value function

$$Q(s, a, \theta) \approx Q^*(s, a)$$

θ are the **function parameters** to be learned.

Instead of taking a as input, it is customary to return a value for each possible action a (the two functions are isomorphic)



The loss function

Given (s, a) the current Q-value estimate of the network

$$Q(s, a, \theta)$$

The expected value, given by the Bellman equation is

$$\mathbb{E}_{s'}[r_0 + \gamma \max_{a'} Q(s', a', \theta)]$$

We try to minimize their distance, to get the fixpoint

$$L(\theta) = (\mathbb{E}_{s'}[r_0 + \gamma \max_{a'} Q(s', a', \theta)] - Q(s, a, \theta))^2$$

Experience replay

For learning, we just need the loss function and its gradient

$$L(\theta) = (\mathbb{E}_{s'}[r + \gamma \max_{a'} Q(s', a')] - Q(s, a, \theta))^2$$

To compute it, we need transitions

$$(s_i, a_i, r_i, T_i, s_{i+1})$$

We can store these transitions in an **experience memory**, and **replay** them at leisure during training (**experience replay**).

Better than learning from batches of consecutive samples because:

- samples tend to be correlated $\Rightarrow$ inefficient learning
- great risk of introducing biases during learning

Q-learning algorithm with experience replay

Playing Atari with DRL, Mnih et al. Nature 2015.

Algorithm 1 Deep Q-learning with Experience Replay

Initialize replay memory $\mathcal{D}$ to capacity N

Initialize action-value function Q with random weights

for episode = 1, M **do**

 Initialise sequence $s_1 = \{x_1\}$ and preprocessed sequenced $\phi_1 = \phi(s_1)$

for $t = 1, T$ **do**

 With probability ϵ select a random action a_t

 otherwise select $a_t = \max_a Q^*(\phi(s_t), a; \theta)$

 Execute action a_t in emulator and observe reward r_t and image x_{t+1}

 Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$

 Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in $\mathcal{D}$

 Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from $\mathcal{D}$

 Set $y_j = \begin{cases} r_j & \text{for terminal } \phi_{j+1} \\ r_j + \gamma \max_{a'} Q(\phi_{j+1}, a'; \theta) & \text{for non-terminal } \phi_{j+1} \end{cases}$

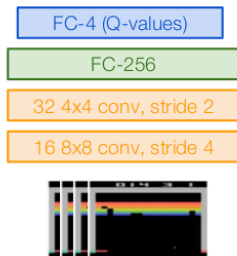
 Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$ according to equation 3

end for

end for

The Atari Q-learning architecture

A single feed-forward pass to compute Q-values for all actions from the current state



Last layer has an output for each action a , returning $Q(s,a)$.

Number of actions between 4-18 depending on Atari game

state: 84x84x4 stack of last 4 frames
(RGB→grayscale conversion, rescaling)



Why stacking frames?

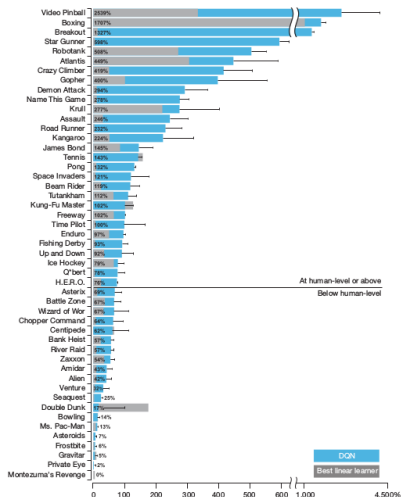
We need a sequence of frames to capture **movement**



As an alternative, you can use a LSTM layer (after processing the state, and before computing Qvalues).



Atari Games and Q-learning



The **same network architecture** was applied to all games

Input are screen frames

Works well for **reactive** games,
not for planning

Recently extended to **Augmented Imagination** (2017)



video



A note on rewards

Rewards are very different from game to game.

How can we use a same architecture?

The solution adopted in **Playing Atari with DRL** was to **crop all positive rewards to 1, and all negative rewards to -1**.

The lesson to be learned seems to be that the **entity of the reward does not really matter**: what matters is to distinguish between positive, negative, and neutral moves.

An alternative: RAM instead of screenshots

The RAM for the Atari 2600 console consists of 128 bytes!



We can directly exploit it as state observations, instead of video screenshots: see [Learning from the memory of Atari 2600](#)

Deep Q-learning improvements:

- Fixed Q-targets
- **Double Q-learning**
- **Prioritized Experience Replay**
- **Dueling**
- Noisy Networks
- Distributional RL
- Rainbow

Fixed Q-targets

Fixed Q-targets

With Q-learning, we try to approximate the optimal target Q-function Q^* , through progressive updates:

$$Q^*(s, a) - Q(s, a)$$

Moreover, we approximate the computation of $Q^*(s, a)$ as $r_o + \max_{a'} Q(s', a')$, giving us the q-learning update

$$\underbrace{r_o + \gamma \max_{a'} Q(s', a')}_{\text{approximated target } Q^*} - \underbrace{Q(s, a)}_{\text{current estimation}}$$

The Q function is computed by a neural network $Q(s, a, w)$.

In the loss function, the *same network* is used to provide two different estimations of the Q-function:

$$\underbrace{r_0 + \gamma \max_{a'} Q(s', a', w)}_{\text{approximated target } Q^*} - \underbrace{Q(s, a, w)}_{\text{current estimation}}$$

At every step of training, the Q value shifts but also the “target value” shifts.

We are getting closer to our target but the target is also moving, that may lead to big oscillations in training.



Fixed Q-targets

Use a separate network $\bar{Q}$ with **fixed parameters** for estimating the TD target.

$$\underbrace{r_0 + \gamma \max_{a'} \bar{Q}(s', a')}_{\text{approximated target } Q^*} - \underbrace{Q(s, a)}_{\text{current estimation}}$$

Periodically, copy the parameters from Q to $\bar{Q}$, to update the target network.

Implementing fixed q-targets is straightforward:

- create two (identical) networks: DQNetwork, TargetNetwork
- define a function to transfer parameters from DQNetwork to TargetNetwork
- during training, compute the TD target using our target network. Update the target network with the DQNetwork every tau steps (tau is a user-defined hyper-parameter).

Double Q-learning

Action values overestimation

The approximation of target action value is computed using a **maximum** over actions:

$$\underbrace{r_0}_{\text{local reward for taking action } a} + \gamma \cdot \underbrace{\max_{a'} Q(s', a')}_{\text{max Q-value over all possible actions}}$$

Since the approximation is noisy, it is possible to prove that this will eventually result in a positive bias, finally resulting in an **overestimation** of the correct value.

Decoupling action choice and its estimation

The **Double Q-learning** approach consists in decoupling the choice of the action from its estimation, using two networks Q^A and Q^B .

1. Initialize Q^A, Q^B, s
2. **repeat**
3. choose a using ϵ, Q^A, Q^B ; observe r, s'
4. choose (e.g. random) between UPDATE-A and UPDATE-B
5. **if** UPDATE-A: # use Q^A to choose action, Q^B to estimate it
6. $a^* = \arg \max_a Q^A(s', a)$
7. $Q^A(s, a) \leftarrow Q^A(s, a) + \alpha(r + Q^B(s, a^*) - Q^A(s, a))$
8. **else if** UPDATE-B: # use Q^B to choose action, Q^A to estimate it
9. $a^* = \arg \max_a Q^B(s', a)$
10. $Q^B(s, a) \leftarrow Q^B(s, a) + \alpha(r + Q^A(s, a^*) - Q^B(s, a))$
11. $s \leftarrow s'$
12. **until** end

Decoupling action choice and its estimation

If we use Q^A to select best action $a^* = \arg \max_a Q^A(s', a)$, the value of $Q^A(s', a^*)$ could be biased by the choice (we know it will be large: it was the maximum!).

Since Q^B was updated on the same problem, but with a different set of experience samples, $Q^B(s', a^*)$ provides a better, **unbiased estimate** for the value of action a^* .

Prioritized Experience Replay

Prioritized Experience Replay

Prioritized Experience Replay (PER) exploits the idea is that some experiences may be more important than others, and thus should be replayed more frequently.

We want to give higher priority to transitions for which there is a large difference between our prediction and the expected target.

For a transition $t = (s, a, r, F, s')$, its update is

$$\delta_t = r + \gamma \max_{a'} Q(s', a') - Q(s, a)$$

and we set its priority to

$$p_t = |\delta_t|$$

The probability of being chosen for a replay is computed according to the following rule:

$$P_t = \frac{p_t^\alpha}{\sum_t p_t^\alpha}$$

If $\alpha = 0$, all transitions have same probability; if α is large, it privileges transitions with high priority p_t .

However, we are introducing a bias toward high-priority samples (more chances to be selected), with a risk of over-fitting over the small portion of experiences that we **presume** to be interesting.

importance sampling weight

We can correct the bias with **importance sampling weights**.

If N is the dimension of the replay buffer, then

$$w_t = (N \cdot P_t)^{-\beta}$$

that compensate the non-uniform probabilities P_t when $\beta = 1$ (if some transition has a high probability, we reduce its weight).

These weights are folded into the Q-learning update by using $w_t \delta_t$ instead of δ_t .

For stability reasons, weights are normalized by $1/(\max_t w_t)$ so that they only scale the update δ_t downwards.

β is initialized to 1 and goes to 0 during training.

Dueling

Advantage

Each Q-value $Q(s, a)$ estimates how good it is to take action a in state s : it depends both on the action a and the value of the given state s .

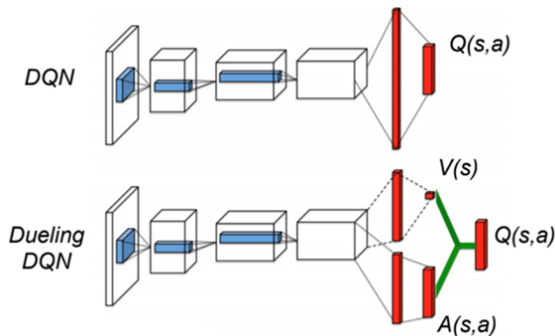
We can decompose $Q(s, a)$ as the sum of:

- $V(s)$: the value of being at state s
- $A(s, a)$: the **advantage** of taking action a in state s , measuring how much better is to take action a versus all other possible actions at that state.

$$Q(s, a) = \underbrace{V(s)}_{\text{value}} + \underbrace{A(s, a)}_{\text{advantage}}$$

Dueling DQN

Dueling Network Architectures (DDQN) split the computation of $V(s)$ and $A(s,a)$ in two different streams:



Intuitively, the dueling architecture can learn which states are (or are not) valuable, without having to learn the effect of each action for each state.

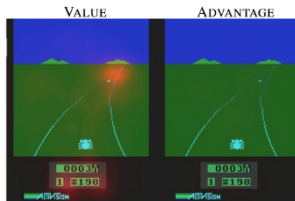
This is particularly useful in states where actions do not affect the environment in any relevant way.

Conversely, in states where the action is relevant, it can focus on the advantage without caring for the current evaluation of the state.

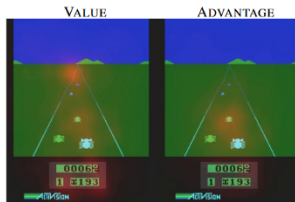
Saliency maps on Enduro

For estimating Value, the network focus on:

- ▶ the horizon
- ▶ the score



no car close by:
action is not relevant



pay attention to
the car in front:
action is crucial

Saliency maps (the orange blurs) are computed with Jacobians (partial derivatives) on input images. See: [Deep Inside Convolutional Networks: Visualising Image](#)

[Classification Models and Saliency Maps](#)



Naif aggregation:

$$Q(s, a) = V(s) + A(s, a)$$

Problem: given $Q(s, a)$, it is impossible to recover $V(s)$ and $A(s, a)$ and hence it is difficult to distribute the error during backpropagation (identifiability).

Aggregating Value and Advantage

$$\cancel{Q(s, a) = V(s) + A(s, a)}$$

Force A to have **zero advantage** for the best action a^* :

$$Q(s, a) = V(s) + A(s, a) - \max_a A(s, a)$$

If $a^* = \operatorname{argmax}_a A(s, a)$, $Q(s, a^*) = V(s)$ and $A(s, a^*) = 0$.

Alternative: mean instead of max

Experimentally, replacing max with mean seem to work better:

$$Q(s, a) = V(S) + A(s, a) - \text{mean}_a A(s, a)$$

Subtracting the mean value helps to improve stability during training.

Noisy Networks

Noisy Networks

Noisy Networks are characterized by **noisy dense layers**, combining a deterministic and a noisy stream:

$$y = \underbrace{b + Wx}_{\text{usual layer}} + \underbrace{(b_{\text{noisy}} \odot \epsilon^b + (W_{\text{noisy}} \odot \epsilon^w)x)_{\text{noisy stream}}}$$

- $W, b, W_{\text{noisy}}, b_{\text{noisy}}$ are learned parameters
- ϵ^b, ϵ^w are randomly generated

This layer is used in substitution of any standard dense layer (doubling the number of parameters).

Random exploration

The purpose of the noise is to augment the randomness in the choice of actions.

Since **noisy-weights are learned**, and the resulting noise is **state dependent**, this allows the network to randomly explore the environment at different rates in different parts of the state space.

Noisy networks replace and seem to work better than the ϵ -greedy strategy.

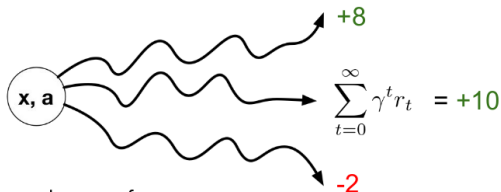
Distributional RL

Distributional RL try to learn the probability **distribution** of the future cumulative reward, instead of the traditional approach of modeling the **expectation** of this return.

Specifically, DRL addresses the random return Z whose expectation is the value Q .

The cumulative reward random variable Z

The r.v. Return $Z^\pi(x, a) = \sum_{t \geq 0} \gamma^t r(x_t, a_t) \big|_{x_0=x, a_0=a, \pi}$



Captures intrinsic randomness from:

- Immediate rewards
- Stochastic dynamics
- Possibly stochastic policy

Slide from **Distributional Reinforcement Learning** by R.Munos.



Distributional Bellman

Let a^* be the best possible action in s' .

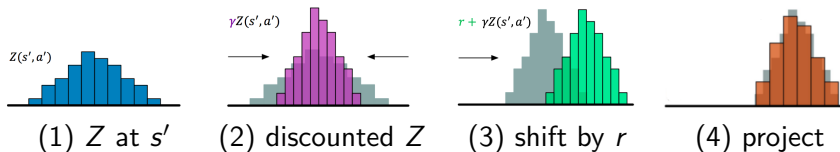
The Bellman equation is

$$Q(x, a) = R(x, a) + \gamma Q(x', a^*)$$

Similarly,

$$Z(x, a) = R(x, a) + \gamma Z(s', a^*)$$

The distribution is discretized over a support in a given range between V_{min} and V_{max} using a fixed number of bins (e.g. 51).



The loss between the two distributions can be computed with KL-divergence.

Algorithm 1 Categorical Algorithm

input A transition $x_t, a_t, r_t, x_{t+1}, \gamma_t \in [0, 1]$

$$Q(x_{t+1}, a) := \sum_i z_i p_i(x_{t+1}, a)$$

$$a^* \leftarrow \arg \max_a Q(x_{t+1}, a)$$

$$m_i = 0, \quad i \in 0, \dots, N-1$$

for $j \in 0, \dots, N-1$ **do**

 # Compute the projection of $\hat{T}z_j$ onto the support $\{z_i\}$

$$\hat{T}z_j \leftarrow [r_t + \gamma_t z_j]_{V_{\min}}^{V_{\max}}$$

$$b_j \leftarrow (\hat{T}z_j - V_{\min}) / \Delta z \quad \# b_j \in [0, N-1]$$

$$l \leftarrow \lfloor b_j \rfloor, u \leftarrow \lceil b_j \rceil$$

 # Distribute probability of $\hat{T}z_j$

$$m_l \leftarrow m_l + p_j(x_{t+1}, a^*)(u - b_j)$$

$$m_u \leftarrow m_u + p_j(x_{t+1}, a^*)(b_j - l)$$

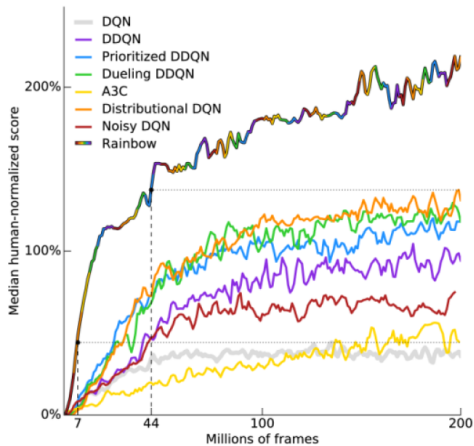
end for

output $-\sum_i m_i \log p_i(x_t, a_t)$ # Cross-entropy loss

Suggested reading: [Distributional Bellman and the C51 Algorithm](#)

Rainbow

Rainbow



DDPG



DQN is a technique addressing discrete action space.

In many cases, actions are continuous.

The Deep Deterministic Policy Gradient (DDPG) is an off-policy actor-critic algorithm generalizing DQN to continuous action spaces.

The Action vector a_t at each timestep t is a vector of floating point values, one for each different action (e.g. acceleration, and steering angle).

DDPG's actor and critic

The DDPG Framework uses two networks:

- the actor $\mu(s|\theta^\mu)$ chooses the actions to take
- the critic $Q(s, a|\theta^Q)$ evaluates the action in a given state

The optimal critic $Q^*(s, a)$, and the optimal policy $\mu^*(s)$ should satisfy the Bellman equation

$$Q^*(s, a) = r(s, a) + \gamma Q^*(s', \mu^*(s'))$$

that allows direct training of the Q-function from transitions (s, a, s', r, T) .

But what about $\mu(s|\theta^\pi)$? We just train it with the objective of optimizing the expected reward:

$$Q(s, \mu(s|\theta^\mu)|\theta^Q)$$

Algorithm 1: DDPG with replay buffer and target networks

Initialize actor $\mu(s|\theta^\mu)$, critic $Q(s, a|\theta^Q)$

Initialize target networks: $\theta^{\mu'} \leftarrow \theta^\mu, \theta^{Q'} \leftarrow \theta^Q$

Initialize replay buffer $\mathcal{R}$

for *each episode* **do**

 Initialize noise process $\mathcal{N}$, receive initial state s_0

for *each timestep* t **do**

 Select action: $a_t = \mu(s_t) + \mathcal{N}_t$

 Execute a_t , observe r_t, s_{t+1}

 Store $(s_t, a_t, r_t, s_{t+1}) \in \mathcal{R}$

 Sample minibatch from $\mathcal{R}$

 Compute target: $y = r + \gamma Q'(s', \mu'(s'))$

 Update critic: minimize $(Q(s, a) - y)^2$

 Update actor: optimize $Q(s, \mu(s | \theta^\mu) | \theta^Q)$

 Soft update targets: $\theta' \leftarrow \tau\theta + (1 - \tau)\theta'$

end